CS405 Project 1 | Draft report

Transformations, Projection and Curves from Paintings
Fall 2026 · 8 November 2026 · Application revision: 90f1731

Painting and scene

Kavunlu Natürmort, Şeker Ahmed Paşa (Sakıp Sabancı Museum collection) was selected for the melon slice's clear organic outline. The painting is a textured quad. Three foreground solids demonstrate transforms; a table with a rotating platform demonstrates hierarchy; curves trace the lower edge of the melon slice. The image was taken from the supplied Week 01 course material.

One Vite application uses raw WebGPU and WGSL. Mathematics, scene traversal, rendering, interaction and UI are separate modules. The common pipeline is $\mathbf{p_{clip} = P\,V\,M\,p_{object}}$. Four views share the application, reset controls and debug switch.

A. Transformations and order

Matrices use column-major storage and act on column vectors; the rightmost transform acts first. The matrix module implements identity, multiplication, translation, scaling, rotations around X/Y/Z, transpose and rigid inverse. For a rigid matrix [R,t;0,1], orthogonality gives the inverse $[R^T,-R^Tt;0,1]$. This shortcut is not valid for a scaled transform.

The middle object has XYZ translation, Y rotation and uniform scale controls. Its rotation also includes a fixed -20° X tilt: $R = R_y(32°)\,R_x(-20°)$ in the captures. With T R S, local geometry is scaled, rotated and then translated. With S R T, the translation vector itself is rotated and scaled, changing the position even though the controls are identical. The rest of the scene uses parent-to-child composition and local T R S ordering where applicable.

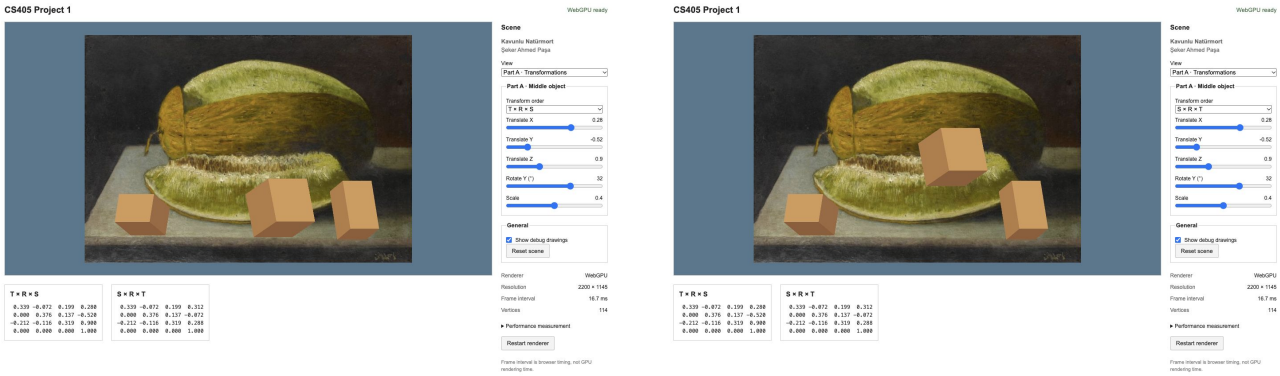


Figure 1. TRS (left) and SRT (right), with identical controls and both numerical model matrices visible. Full-resolution captures are stored in docs/screenshots/.

```
TRS origin -> (0.280, -0.520, 0.900)
SRT origin -> (0.312, -0.072, 0.288)
Defaults: t=(0.28,-0.52,0.90), scale=0.40, Y angle=32 deg.
```

The translation column directly verifies the origin's image because multiplying by (0,0,0,1) selects that column. A direction has homogeneous w = 0, so translation does not change it.

B. Scene graph and a leaf world matrix

Each node stores its local transform, parent, children and world matrix. Traversal computes $W_{child} = W_{parent}L_{child}$. There are nine nodes across four levels, including the root. Rotating the turntable carries both blocks without separately editing their world positions.

```
Scene
  Table
    Front left leg     Front right leg
    Back left leg      Back right leg
    Turntable
      Small block
      Tall block
```

The table and blocks have separate mesh-sizing matrices. These affect the geometry but do not scale the frame inherited by a child. The rendered model is $W_{node}S_{mesh}$; local debug axes use W_{node}. This keeps a thin tabletop from shrinking its descendants.

Numerical verification at the reset pose

For the small block, angles are in radians and the animation is paused at the reset angle 0.28. The complete chain of local frames is:

```
W = T(0,−0.28,1.03) Rx(0.32) T(0,−0.24,0)
    T(0,0.105,0) Ry(0.28) T(−0.27,0.13,0)
```

```
W_small =
  [  0.96106  0.00000  0.27636 −0.25948 ]
  [  0.08693  0.94924 −0.30232 −0.30822 ]
  [ −0.26233  0.31457  0.91227  1.09926 ]
  [  0.00000  0.00000  0.00000  1.00000 ]
```

By hand, the block origin first becomes (-0.27,0.13,0). The turntable rotation sends it to (-0.27 cos(0.28),0.13,0.27 sin(0.28)). The table and turntable translations leave y = -0.005 before the root rotation. Applying $R_x(0.32)$ and the root translation yields **(-0.25948, -0.30822, 1.09926)**, matching the last column above to rounding. Mesh scale diag(0.19,0.16,0.19,1) is applied only after this frame is obtained.

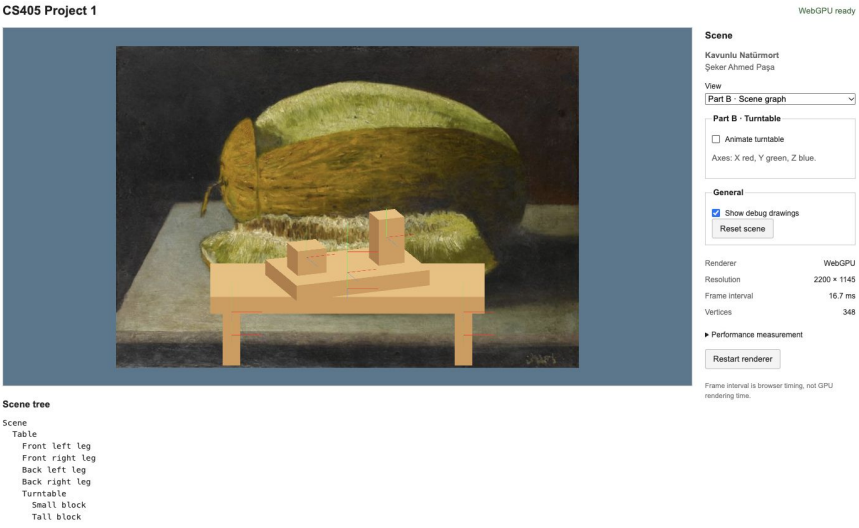


Figure 2. Paused hierarchy with local axes and scene tree. This capture is not the reset-pose matrix calculation; the turntable had already moved before it was paused.

C. View and projection

Deriving lookAt

For eye e, target a and reference up h, define the backward unit axis w = normalize(e - a), the right axis u = normalize(h × w), and v = w × u. Taking coordinates relative to e gives x_{view} = u·(p-e), and similarly for v and w. Therefore the rows of the view matrix are:

```
V = [ ux uy uz −dot(u,e) ]
    [ vx vy vz −dot(v,e) ]
    [ wx wy wz −dot(w,e) ]
    [  0  0  0     1     ]
```

If eye and target coincide, the implementation rejects the camera. If up is parallel to w, their cross product cannot define u; an alternate reference axis is selected before normalization. Mouse orbit updates eye around the target. The initial eye is approximately (0.45,0.61,5.04), target (0,-0.12,0.52).

WebGPU depth convention

View space looks down -Z. Let f = 1/tan(FOV_y/2), aspect r, near n and far F. The matrices below are written as rows, although stored column-major:

```
Perspective:                    Orthographic (half-height h):
[ f/r  0    0          0 ]      [1/(hr)  0     0         0 ]
[  0   f    0          0 ]      [  0    1/h    0         0 ]
[  0   0  F/(n−F)  Fn/(n−F)]    [  0     0  1/(n−F) n/(n−F)]
[  0   0   −1          0 ]      [  0     0     0         1 ]
```

At z = -n, perspective division gives z_{NDC} = 0; at z = -F it gives 1. Orthographic has w = 1 and the same endpoints. Perspective w = -z causes foreshortening; orthographic size is independent of distance. With n = 0.65 and F = 6.5, F/n = 10. Keeping near away from zero improves perspective depth precision while preserving the scene; reducing far can further limit the covered range. Precision remains nonuniform in perspective.

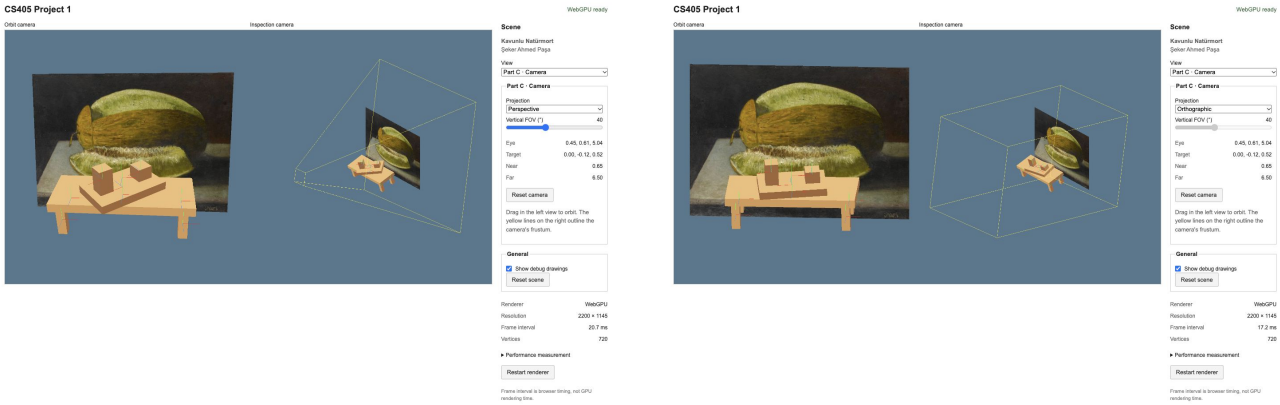


Figure 3. Perspective (left) and orthographic (right) at the same eye and target. Each capture includes the fixed inspection camera and yellow frustum. The animated blocks change orientation between captures.

The frustum is built from four corners on each near/far plane, transformed back to world space with V inverse, and connected by twelve edges. The inspection camera then projects those world-space lines. This makes the difference between a perspective frustum and an orthographic box visible independently of the main view.

D. Curves traced over the melon

Cubic basis, endpoints and de Casteljau

For a segment with control points P_0 through P_3, the Bernstein weights are $(1-t)^3$, $3(1-t)^2t$, $3(1-t)t^2$ and t^3. The curve is their weighted sum:

```
B(t) = (1−t)^3 P0 + 3(1−t)^2 t P1
     + 3(1−t)t^2 P2 + t^3 P3,     0 <= t <= 1
B'(t) = 3(1−t)^2(P1−P0) + 6(1−t)t(P2−P1)
      + 3t^2(P3−P2)
```

At t = 0 the weights become (1,0,0,0), so $B(0) = P_0$; at t = 1 they become (0,0,0,1), so $B(1) = P_3$. De Casteljau repeatedly interpolates adjacent points $Q_i = (1-t)P_i + tP_{i+1}$, reducing four points to three, then two, then one. The final point equals B(t); the intermediate levels are drawn for the active segment.

Three segments and C1 continuity

Ten control points define segments [P0..P3], [P3..P6] and [P6..P9]. Adjacent curves share an endpoint (C0). For equal segment durations, C1 requires equal endpoint derivatives: 3(P3-P2) = 3(P4-P3), hence **P4 = 2P3-P2**; likewise P7 = 2P6-P5. Initial joins satisfy P3-P2 = P4-P3 = (0.18,-0.01) and P6-P5 = P7-P6 = (0.05,0.10).

Dragging an interior handle mirrors its partner about the shared anchor; dragging an anchor also moves its neighboring handles. Moves that would put a point or its mirrored handle outside [0,1] are rejected. The global parameter assigns equal time to each segment. Animation uses the derivative direction for the red arrow; uniform parameter steps do not imply constant arc-length speed.

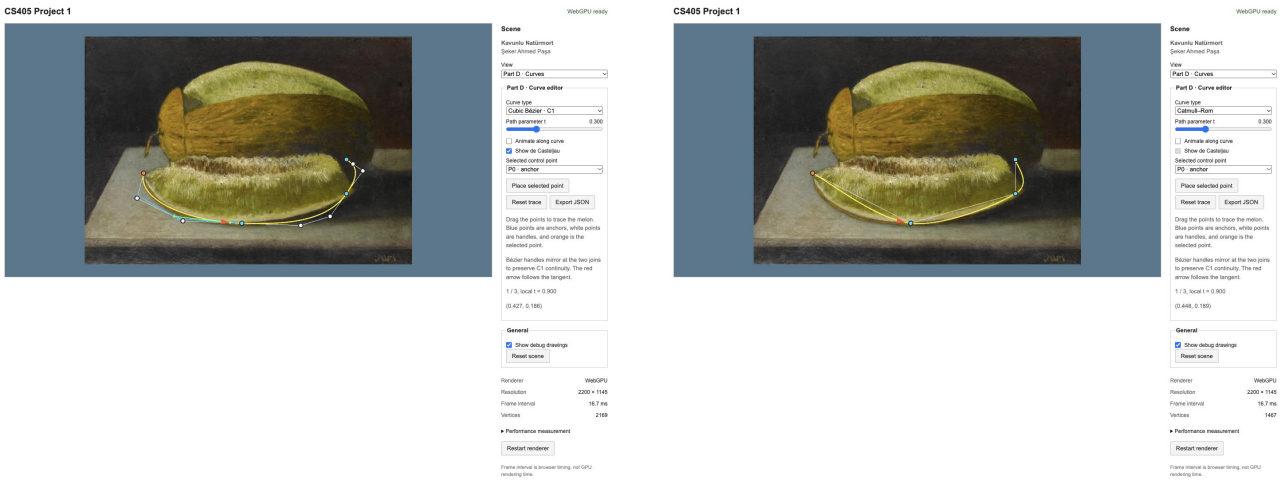


Figure 4. C1 Bézier with de Casteljau (left) and uniform Catmull-Rom through the anchors (right). Both are paused at global t = 0.300; the Bézier segment-local t is 0.900.

Second family and export

Uniform Catmull-Rom uses four consecutive knots and interpolates the middle pair; neighboring segments share the central-difference tangent. Endpoint knots are repeated. It reuses the four Bézier anchors, so it passes through those anchors without following the Bézier handles. Coordinates are normalized to the painting with a bottom-left origin. Appendix A reproduces the exported data; the full file is exports/curve.json.

E. Integration, measurements and appendices

The View selector switches A-D without changing pages. Reset scene restores transforms, camera, trace and debug defaults. The production bundle builds with npm run build and is served with npm run preview; normal development uses npm install followed by npm run dev. Browser inspection found no captured warning/error messages during the final evidence session.

Measured performance

Apple M2 (10 GPU cores), macOS 26.6.2, Chrome 156.0.8078.80; canvas 2200 × 1145 backing pixels, DPR 2. Each retained run has a 3 s warm-up and at least 30 s of visible-tab sampling. Hierarchy/camera animation is on; the curve run uses Bézier animation, debug and de Casteljau. All foreground intervals, including stalls, are retained; hiding or resizing cancels recording.

Scene	Mean ms	P95 ms	Max ms	FPS	CPU ms	Vertices
A	16.67	17.70	17.80	60.00	0.65	114
B	16.69	17.70	33.50	59.93	0.80	348
C	16.87	17.70	149.40	59.27	0.95	720
D	16.69	17.60	67.30	59.90	2.34	2169

FPS = 1000/mean interval; P95 uses nearest rank. CPU time covers JavaScript rendering work through queue.submit; neither CPU time nor browser frame interval measures GPU execution. Near-60 FPS pacing limits conclusions about throughput. Camera submits both views; curve markers and construction produce more vertices. Camera's 149.4 ms maximum shows a stall hidden by the mean. One retained run per scene is not a multi-device benchmark. Raw samples and conditions are in docs/measurements/.

Debugging and practical limitations

The local preview became unavailable when its serving process stopped; restarting the server restored it. During evidence capture, waiting for a download timed out even though the JSON had been saved. Checking the actual files and reopening a browser tab recovered the workflow. Early runs used a different canvas size, so A/B were repeated at the common resolution; exploratory files are kept separately. The camera comparison keeps the camera pose fixed, but the turntable moves between captures. No GPU timestamp queries or repeated-run variance analysis were performed.

Appendix A. Exported curve (compact JSON)


```
{"painting":"Kavunlu Natürmort","artist":"Şeker Ahmed Paşa",
 "type":"piecewise-cubic-bezier",
 "coordinates":{"units":"normalized","origin":"bottom-left",
                "xRange":[0,1],"yRange":[0,1]},
 "segments":3,"continuity":"C1","controlPoints":[
 {"x":0.18,"y":0.40},{"x":0.16,"y":0.29},{"x":0.30,"y":0.19},
 {"x":0.48,"y":0.18},{"x":0.66,"y":0.17},{"x":0.75,"y":0.21},
 {"x":0.80,"y":0.31},{"x":0.85,"y":0.41},{"x":0.82,"y":0.44},
 {"x":0.80,"y":0.46}]}
```


Appendix B. AI assistance and sources - draft status

OpenAI Codex assisted with requirements review, planning, implementation of application code and mathematics, documentation, debugging, evidence capture, measurement tooling and this report. The student stated that the course AI-use rules had changed to allow this assistance and will maintain the detailed usage record. That student-maintained record and final wording still require review before submission; this draft does not claim independent authorship of generated code.

Sources: CS405_Project1_2026.docx (requirements); supplied Week 01 painting material; repository source, exports/curve.json, docs/screenshots/ and docs/measurements/. Repository: github.com/efe-akkus/cs405-project1. This is a review draft, not the final submission PDF.